PROJECT PLANNING DOCUMENT

Habit Tracker

Under the heading "Idea," submit a written document for

- Project Team
- Project Information and Scope
- Project Risks, Constraints, and Feasibility

1. PROJECT TEAM

Describe how the project will be organized and managed. Identify reporting lines and outline specific roles that will be filled. You need to be clear about staff roles so that you don't duplicate responsibilities and everyone is clear about what's expected of them. If this is a long-term project, you may even consider developing job descriptions for team members.

Team Member	Responsibilities
Andrew Dang	Web Design, Frontend
David Salgado	Database, Backend

2. PROJECT INFORMATION AND SCOPE

Habit Tracker

This project is a web-based habit tracker and calendar app designed to help people better understand and improve their daily routines. The main idea is to turn everyday actions, whether they're good or bad, into measurable statistics that are easy to keep track of. Users can log different habits, such as making the bed, procrastinating, cooking, or anything else they do regularly. Both positive and negative habits can be recorded, which helps users get a more complete picture of how they spend their time.

The app features a "stats page" where users can see their progress over time. It shows detailed charts and trends based on the habits they've logged, breaking them down by day, week, and year. This makes it easier to spot patterns and see where improvements can be made. Whether someone wants to build better habits, stop bad ones, or just understand their daily life better, this tool gives them the data they need to stay motivated and make smarter choices.

Target Audience: Young adults who are willing to improve their daily lives.

Unlike other existing habit trackers, this project is noteworthy because it goes beyond a simple calendar or to-do list—providing analytics of life patterns through real data, offering users a deeper understanding of their daily behavior and unique insights into how they organize their lives and habits.

3. PROJECT RISKS, CONSTRAINTS, AND FEASIBILITY

With proper time management, a clear strategy, and no extreme setbacks, this project will be completed on time.

The project does not currently integrate with other systems, as it is designed to be a self-contained web application. Unlike existing habit trackers such as Google Calendar add-ons or mobile apps, this project focuses on analytics of life patterns rather than simple reminders or streaks. It stores data independently using JSON Bin or NoSQL with JavaScript, ensuring consistency and reliability without depending on third-party integrations. To our knowledge, no current habit tracker combines both positive and negative habit logging with long-term statistical analysis in the way this project proposes.

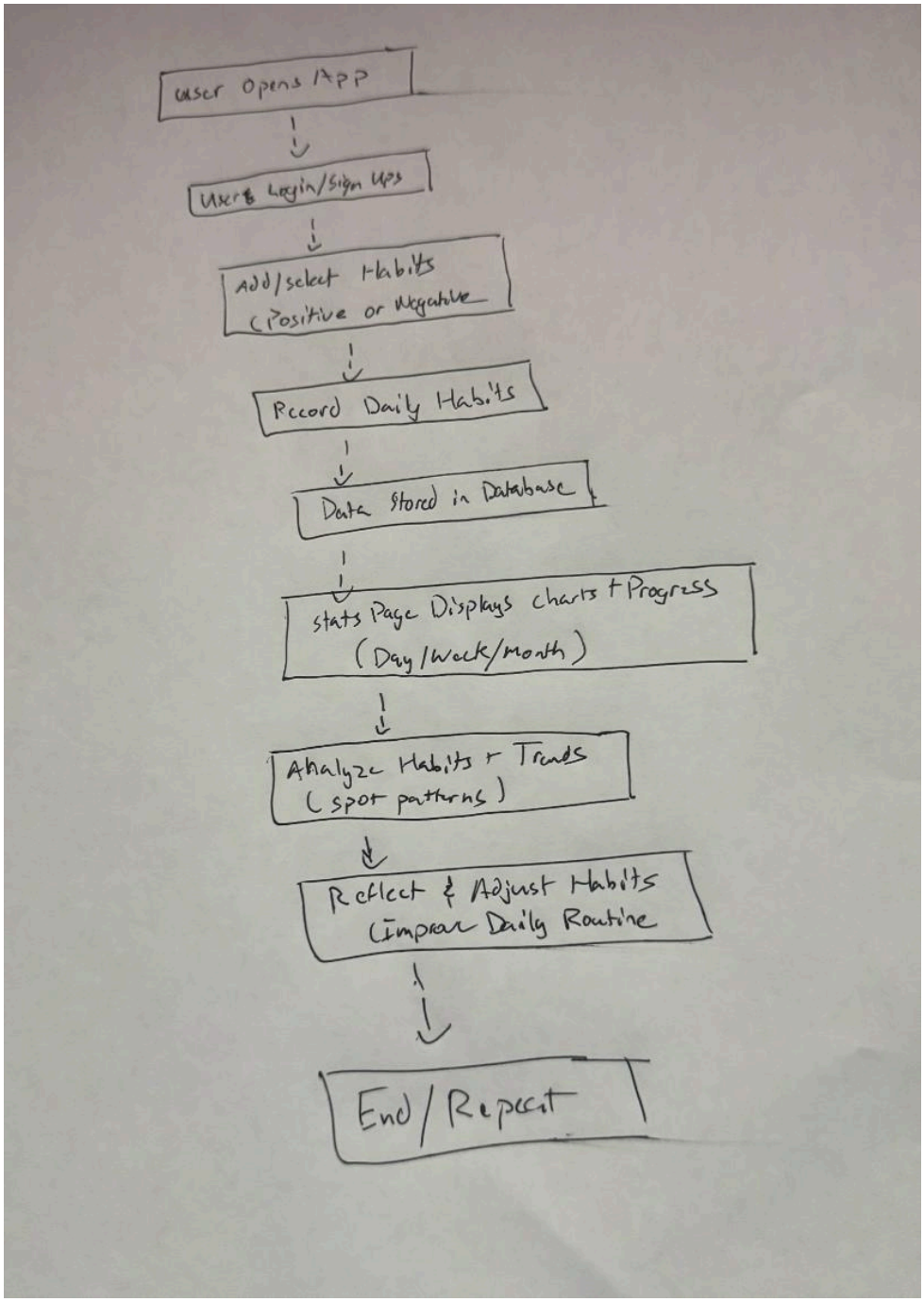
Some potential limitations of this project include maintaining user motivation and consistency, as well as challenges in designing a back-end scoring system that fairly evaluates the value of habits as 'good' or 'bad.'

The major risk is the scoring system not outputting correctly, since all the stats are heavily dependent on how the input gets stored. To mitigate this risk, we will focus on having simple formulas at first, test them with sample inputs, and adjust the weights and polarity so the model gets fine-tuned.

Not that we can think of at the moment.

The most critical is the categorization engine, because it determines how habits are classified. This categorization directly drives the statistical HUD and all its output values.

4. VISUALIZE THE PRODUCT: MAP THE PROJECT ARCHITECTURE



5. Project Methodology

Our team will keep track of progress by meeting weekly to discuss what has been done, any issues that arose, and next steps. We will use Google Docs to organize tasks and show what's finished or still in progress. GitHub will help us save and track changes to our code. We will also write short updates about their work and any issues they faced. By staying organized and communicating often, our team can handle challenges and keep the project moving forward.

6. PROJECT LANGUAGE AND TOOLS: ARCHITECTURE REQUIREMENTS

Product Requirements:

- Any device with a modern web browser (laptop, desktop, tablet, or smartphone

Inputs:

- User Actions (habit creation, deletion, updates)
- Form Data (habit name, type, frequency, time logged)
- Credentials (login/signup information)

Input and Output Devices:

- Keyboard, Mouse, Touchscreen (Mobile or Tablet)

Output Devices:

- Monitor, Mobile Screen, or any display with browser capability

Servers with Description:

- Web Server: Hosts frontend and serves static files
- Application Server: Handles API requests, user authentication, and logic
- Database Server: Stores user data, habits, and logs
- MAMP LocalHost: Functions like a temporary local server for testing.
- JSON Bin Servers: Cloud services providing places to store data in JSON format.

Database Requirements:

Type: Relational Database

- User Profiles
- Habit Entries
- Analytics summaries and statistics
- JSON Bin

Hosting:

- Database remains hosted externally by JSON Bin services.

Development and production environments:

- Development on local MAMP environments and VS Code.
- Production via a hosted website connected to a live JSON Bin database.

Interfaces or Integrations with other hardware: N/A

Software requirements:

- Web browsers supporting ES6 JavaScript.
- Visual Studio Code.
- MAMP (for local hosting).
- Internet access for JSON Bin API calls.

Capacity, space requirements:

- JSON Bin storage is minimal (lightweight JSON files per user).
- Browser caching handles short-term storage.

7. PRODUCT FEATURES AND ENTITIES

1. **Storyboard the program. To include**

    a. User interface(s)

i. **Text/content:**

The initial display will be the Calendar and the title of the app in the header with a background image. The dashboard will have a create account and login buttons that display modal windows. After logging in, the "Create Account" button will transform into the "Add Habit" button, which will also open a modal window for the user to input.

ii. **Visuals/media:**

The background of the dashboard will be a wallpaper media photo. In the future, a logo will be added to the main UI.

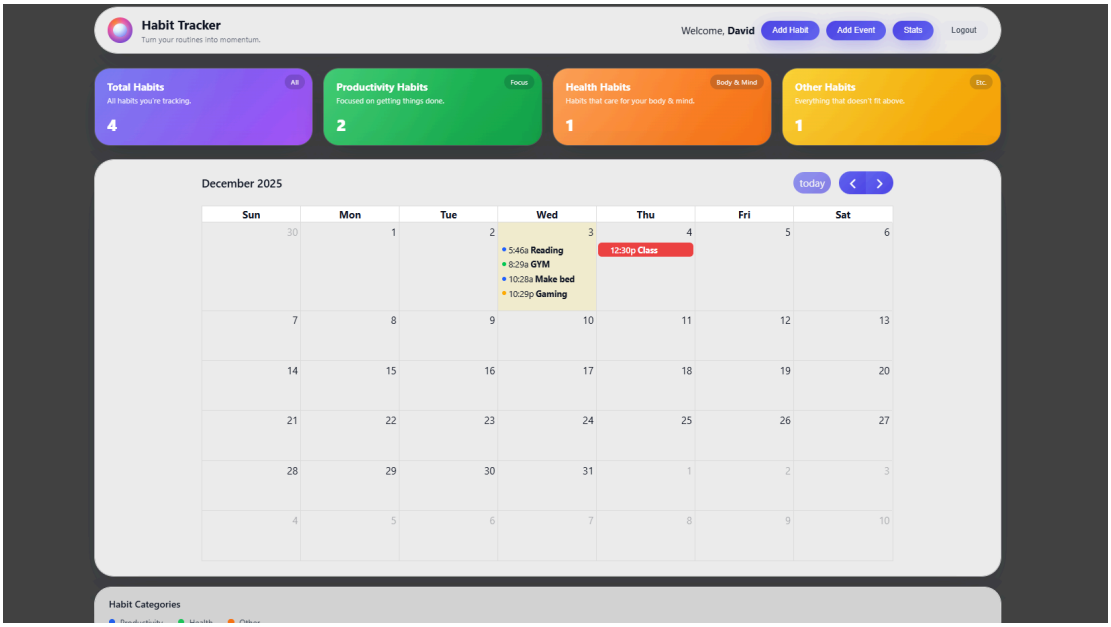
b. **User inputs and interactions**

- Users can create an account or log in.
- After logging in, users can add, edit, or delete habits.
- Each habit can include a title, category, due date, and optional time span.
- Users can mark habits as complete, which updates the calendar and visual indicators.

c. **Expected outputs or results**

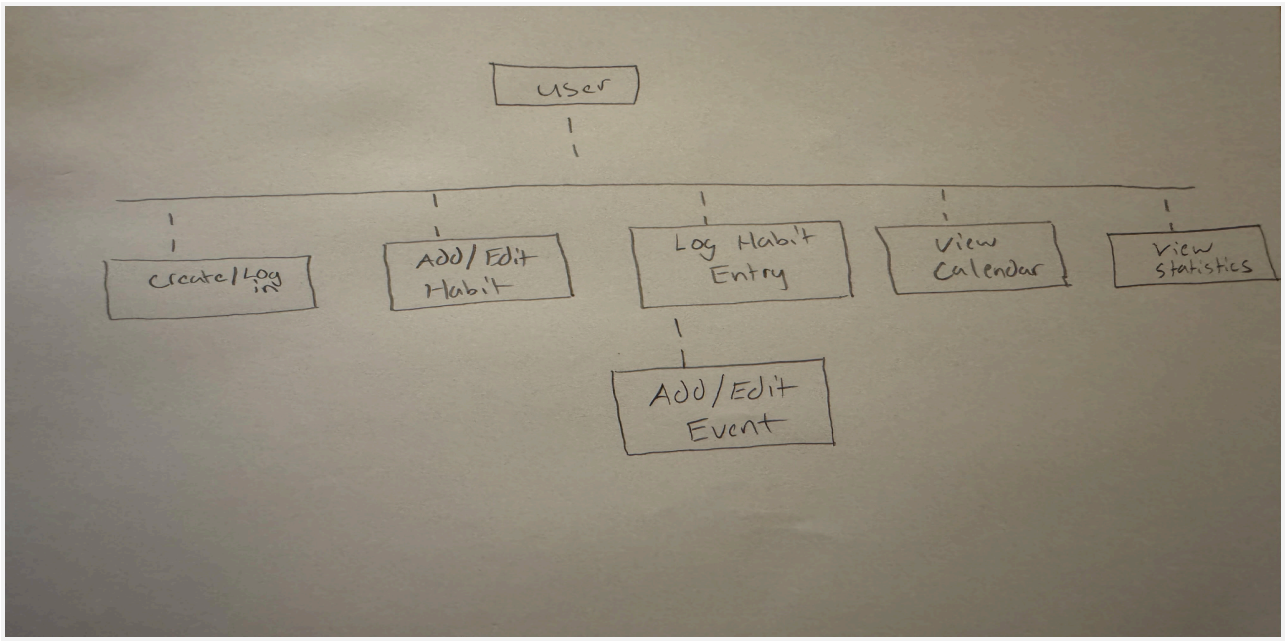
- Display of events alongside daily and weekly habits on the calendar.
- Confirmation messages for actions like saving or deleting habits.
- Buttons and Modal windows for creating an account. Log in and add a habit.

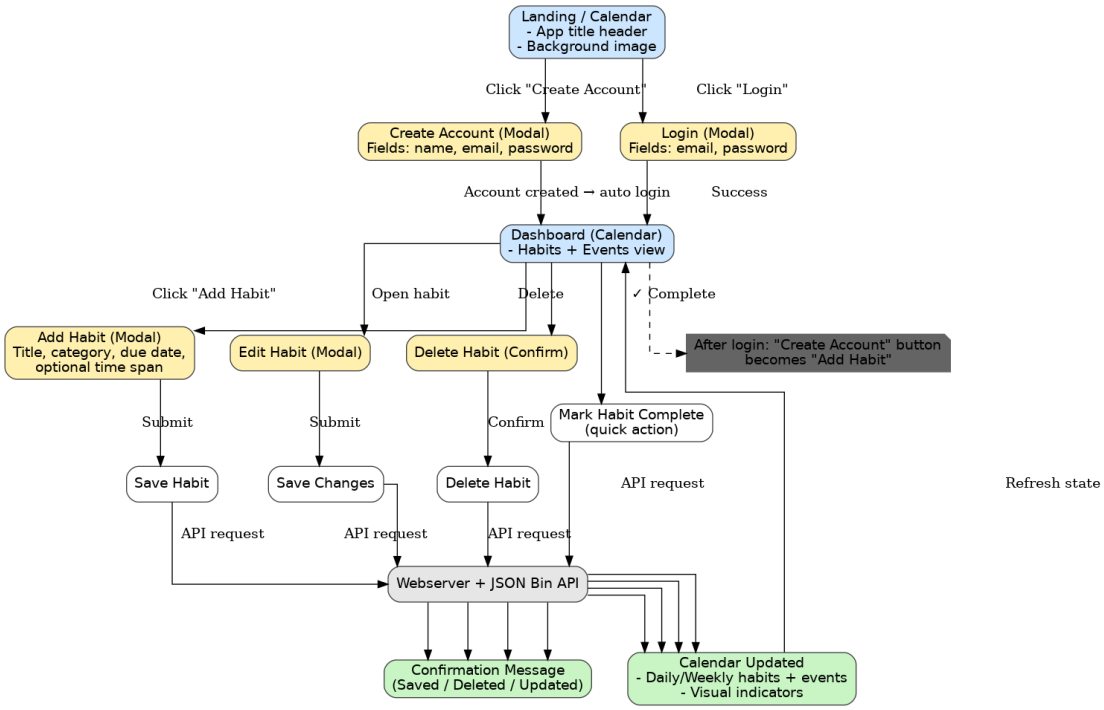
d. **Interface design**



8. MAP THE PROJECT ARCHITECTURE: USE CASE DIAGRAM

1. Create a Use Case diagram for your product





9. LIST ALL THE HIGH-LEVEL FUNCTIONS AND FUNCTIONAL REQUIREMENTS

1. User Account Management
2. Habit Management
3. Habit Logging
4. Event/Calendar Logging and Management
5. Calendar Viewing
6. Statistics Page
7. Trend Visualization and Insights

10. LIST ALL USER INTERACTIONS

User Inputs

- Enter username and password
- Type habit name
- Choose habit category (list)
- Type event name/description
- Select date/time for habits or events
- Enter/edit habit or event detail.

User Interactions

- Click “Add Habit”
- Click “Log Habit”
- Click “Add Event”
- Click to edit or delete habits/events
- Navigate calendar (next/previous month)
- Click charts or filters on the stats page
- Click the “stats” button to view the pie chart.

11. DATA MANAGEMENT

1. The Web Application doesn't use traditional databases such as SQL servers; instead, it connects to a cloud JSONBin.io through HTTPS. Each bin in JSON functions as a small NoSQL database. The web app sends “GET” and “PUT” requests via JavaScript to read and push in data into the database.
    a. Currently, the web application uses three main bins:
        i. ***Users:*** *Storoes login credentials*
        ii. ***Habits:*** *Stores users' habit input*
        iii. ***Events:*** *Stores user input event.*

2. **Types of data stored:**

    a. HABIT BIN
        i. ***id****: Habit input identifier string.*
        ii. *userName: User-chosen string for username*
        iii. ***Name****: string of the habit defined by the user*
        iv. ***Category:*** *Label used for filtering and analytics.*
        v. ***Time:*** *Chosen time when storing habit in “HH: MM” format.*
        vi. ***Active:*** *Boolean to indicate whether the habit is active or inactive. Used for soft deleting.*
        vii. ***createAt:*** *Timestamp of input.*

    b. LOGIN BIN
        i. ***Id:*** *A string unique identifier assigned to each created user name. Used to link users to their habits and events on other bins.*
        ii. ***userName:*** *String created as username. This field is visible to the interface and can be used to filter related data.*
        iii. ***Password:*** *The user's password. In this prototype implementation, passwords are stored in plain text for simplicity.*

    c. EVENT BIN
        i. ***id:*** *used to identify the event in the database uniquely.*
        ii. ***title:*** *Descriptive string chosen by the user for the event.*
        iii. ***date:*** *The scheduled date of the event stored in "YYYY-MM-DD" format, allowing compatibility with calendar libraries.*
        iv. **time:** The event time stored in "HH: MM" format.
        v. **createAt**: Timestamp of input.

3. **Data types, archiving, warehousing, and Structure**

a. All three bins use simple JSON data, including strings, booleans for status flags and functions, arrays for record collections, and objects for individual records. Since this is a small prototype, there is no dedicated archive or data warehouse. All records remain in the JSON document format stored in JSONBin.
b. The database structure wirks by having each bin one top level array, and each array contains JSON objects. There is no tables or schemas needed, the application reads and updates each document and entry using HTTP GET and PUT requests through JSONBins cloud services

12. HOW IS SUCCESS DEFINED?

1. **How do you define success?**

    a. Success can take many different forms. I like to define it as the continuous process of becoming a better version of myself while moving toward a dream or goal that feels meaningful. I see myself as a system, and success is the navigation of that system toward the future I envision. To do that, I have to go deep into the small layers of my own patterns, thoughts, emotions, and the way I distribute my energy. By looking at myself on that microscopic level, I start to understand the patterns that shape my behavior, which gives me data I can use as a metric of differentiation, who I am today versus who I want to become. With that awareness, I can make educated choices that reduce sources of chaos or stress and help me change harmful patterns into healthier ones. So success represents the journey itself.

1. **Are there any metrics that can be defined that measure success?**

    a. When it comes to measuring success, the overall idea of defining universal metrics is almost impossible because success is extremely relative. People measure it through many different lenses, such as money, social status, professional achievements, spirituality, religion, or personal fulfillment, so creating a single set of metrics for all humans would never capture what success truly means to each individual. However, I do think that one meaningful metric that goes beyond external markers is the level of balance and harmony in a person's life. If someone has organized the different layers and components of their life in a way that produces a sense of peace, ease, and confidence, that reflects a deeper form of success. A perfect life doesn't exist, but a life where the rate of chaos is significantly lower, where a person feels grounded within themselves, might be the closest thing to a universal measure of success.

1. **Explain any processes or plans that ensure product quality. (i.e. code and product testing)**

a. To ensure the success of the product, it is important to run multiple tests that help identify and mitigate both small and large errors within the system, as well as maintain organized code. Our quality strategy begins with a simple approach: adding features gradually, one at a time, to ensure smooth scaling and prevent major structural issues. This incremental process allows us to test each component individually before integrating everything into a larger system. Another essential part of our quality process is seeking feedback from peers. As developers, we may have unconscious biases that influence our decisions, which can be either beneficial or harmful. To reduce this risk, we regularly ask peers to review our code, interface, and overall design of the web application. This collaborative feedback loop is a key step in ensuring a reliable and high-quality product.

13. CODE AND PRODUCT TESTING

1. **Code builds with no errors:**
    a. For each Increment, we ran the application in a local host web browser to verify any syntax errors or missing imports. The goal is to make sure the project loads correctly, the JavaScript files compile, and the calendar renders without any warnings.
    b. Secondly diving the code into multiple files to prevent large code.
    c. Lastly, every small change or addition to the code was immediately tested before creating more structure and functions.

2. **Code peer review;**
    a. There was no formal peer review process using GitHub or pull requests. Instead, my teammate and I shared files through Discord and discussed changes directly. We occasionally reviewed each other's code informally when bugs appeared, but there was no structured checklist or approval before merging. In the future, we plan to use GitHub pull requests for clearer teamwork and version control.

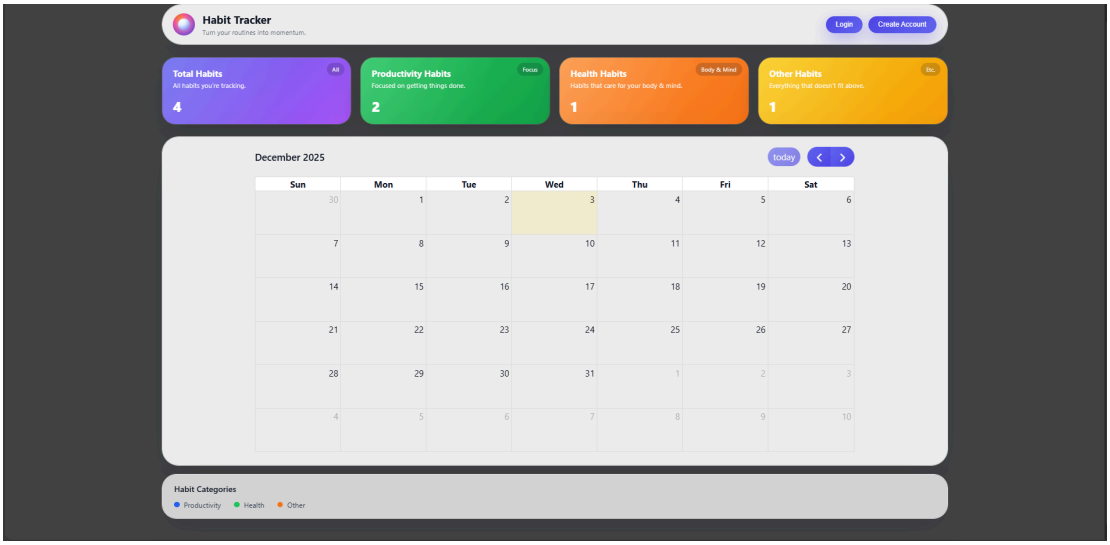
3. **Documentation/comments**
    a. The code will include multiple comments sections to properly outline chunk of code based of the functionality they are providing. Also, small comments will be added to specific [arts of the code to simplify discrete functionalities.

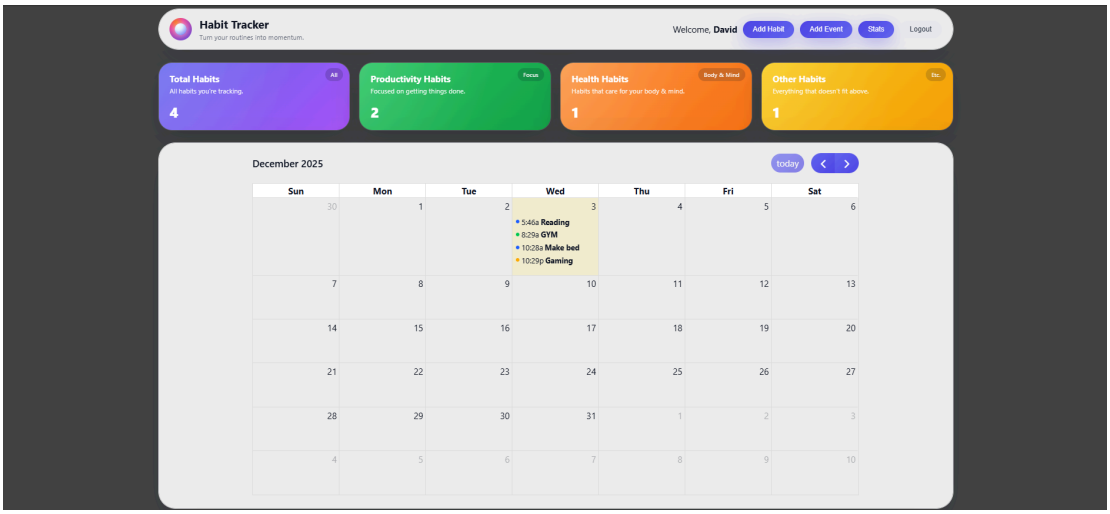
4. **Functionality testing**
    a. For critical functions, such as saving data, filtering habits, and updating the UI, we test them individually. This means calling the function with different data and checking if the result is correct.
    b. Testing sample data in and out of the bin before adding functions was a crucial step.

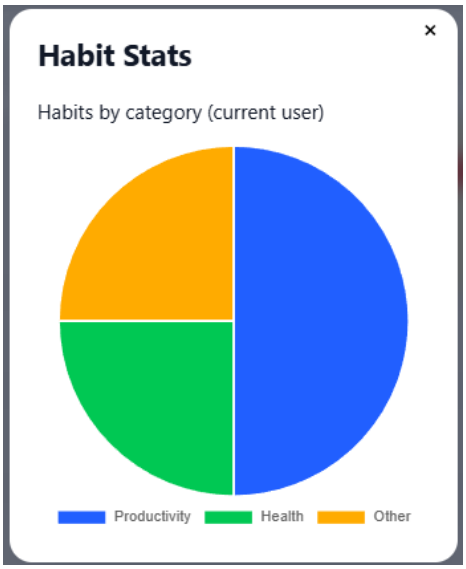
5. **Valid and invalid inputs**
    a. The app will be tested with both valid and invalid inputs. For example, trying to save a habit without a name, using duplicate habit names, entering a date in the past, or leaving fields empty. The app should handle invalid input by showing error messages instead of crashing.

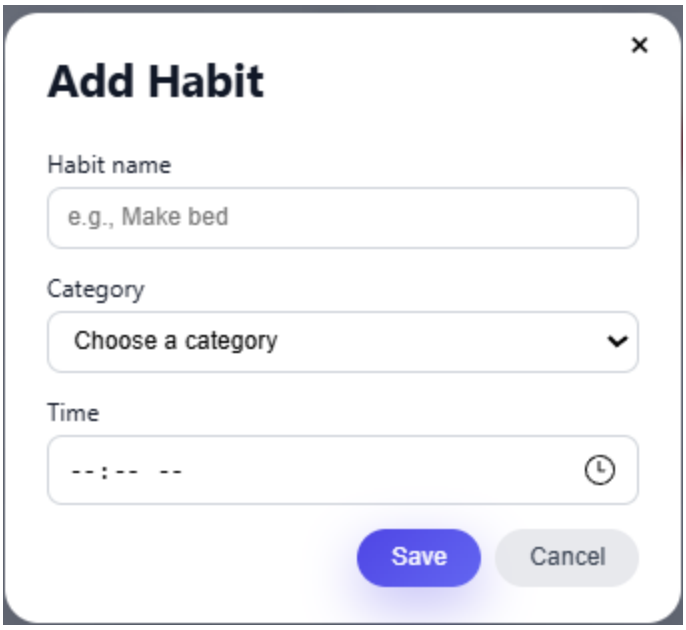
6. **Performance testing**
    a. Observations of how the app behaves when there is more data, such as many habits or events. This includes checking that the calendar remains smooth, that loading from JSONBin is fast, and that the UI does not freeze. If performance drops, I will optimize the data filtering logic.

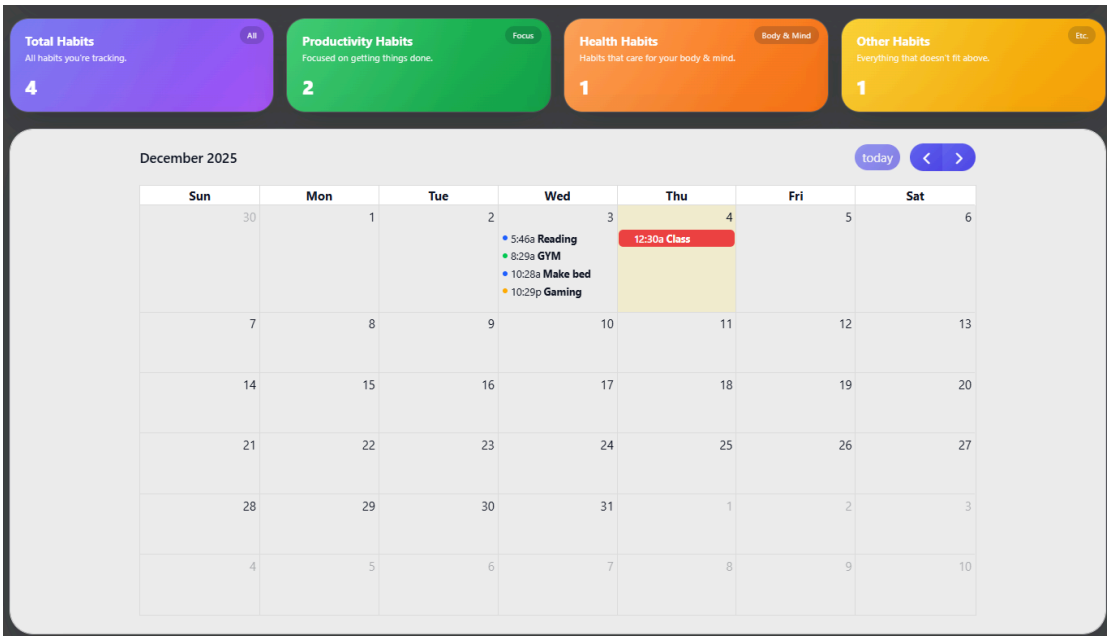
14. FINAL PRODUCT SUBMISSION AND DEMONSTRATION






Habit Stats
Habits by category (current user)
Productivity
Health
Other


Add Habit
Habit name
e.g., Make bed
Category
Choose a category
Time
--:-- --
Save
Cancel


Total Habits
All
All habits you're tracking.
4
Productivity Habits
Focus
Focused on getting things done.
2
Health Habits
Body & Mind
Habits that care for your body & mind.
1
Other Habits
Etc.
Everything that doesn't fit above.
1
December 2025
today
Sun
Mon
Tue
Wed
Thu
Fri
Sat
5:46a Reading
8:29a GYM
10:28a Make bed
10:29p Gaming
12:30a Class